KAVACH

Detailed System Architecture & Engineering Guide

Graph Neural Network Corporate Financial Risk Intelligence Platform

- 15 architecture diagrams, broken down by subsystem
- Mobile · Backend · Risk Engine · ML pipeline · Graph data model
- Explainable AI · Network contagion · Scenario & portfolio
- Voice & multilingual pipeline · Security · Deployment
- Grounded in 20+ research papers (see docs/RESEARCH.md)
- Rapid-fire Q&A and “claims to avoid” guardrails

Built from the project source. Read top to bottom once, then use the diagrams as recall anchors.

Contents

1	What Kavach Is
2	System at a Glance — High-Level Architecture
3	The Core Insight — Why a Graph
4	Mobile-Side Architecture
5	Backend / API Layer
6	The Risk Engine — Inference & Analytics Core
7	ML Pipeline — Data → Features → Graph → Models
8	The Models — Baselines vs GCN / GraphSAGE / GAT
9	Handling Class Imbalance
10	Explainable AI — Feature & Network Attribution
11	Network Analytics — Contagion, Centrality, Simulation
12	Scenario & Portfolio Intelligence
13	Data Model — PostgreSQL + Neo4j
14	Voice & Multilingual Pipeline
15	Security & Resilience
16	Deployment
17	Trade-offs & Known Weaknesses
18	Interview Mental Model (Master Diagram)
19	Claims to Avoid Unless Verified in Code
20	API Route Catalog
21	Rapid-Fire Q&A

1 What Kavach Is

Kavach (Sanskrit for *shield*) is a **Corporate Financial Risk Intelligence Platform** that predicts corporate **financial distress** — the state preceding default, restructuring, insolvency and bankruptcy — using **Graph Neural Networks**. Its defining idea: a company's risk cannot be read from its balance sheet alone; it depends on the health of the companies it is connected to (suppliers, customers, subsidiaries, lenders, peers).

Core capabilities

- Distress probability + risk band (LOW / MODERATE / HIGH / CRITICAL)
- Multi-horizon prediction (3 / 6 / 12 / 24 months)
- Network contagion, centrality, supplier/customer concentration
- Explainable AI — feature drivers AND which connected companies drove the score
- Scenario stress testing and supplier-failure network simulation
- Portfolio distress exposure with shared-network concentration
- Early-warning alerts, industry risk heatmap, multilingual voice assistant

Technology stack

- **Mobile:** Flutter + Dart, Provider, Dio, fl_chart, graphview
- **Backend:** FastAPI (thin routers) → Risk Engine → Postgres / Neo4j
- **ML:** PyTorch + PyTorch Geometric (GCN/GraphSAGE/GAT); scikit-learn + XGBoost
- **Explainability:** SHAP, GNNExplainer / PGExplainer, GAT attention
- **Infra:** Docker, docker-compose, GitHub Actions, Render / Neo4j Aura

2 System at a Glance — High-Level Architecture

Four layers: a **Flutter client** talks to a **FastAPI route layer**, which delegates to a **Risk Engine** owning all modelling and analytics, which reads from **PostgreSQL + Neo4j** (or a self-contained synthetic economy in demo mode).

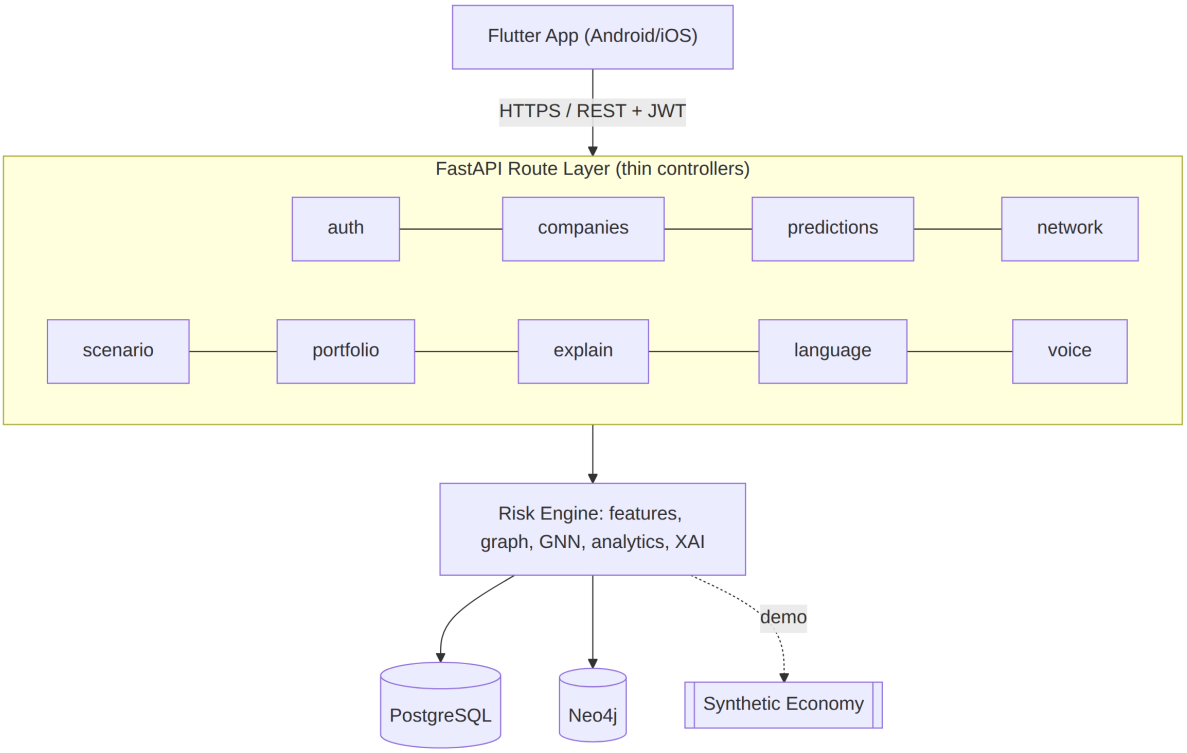


Diagram 1 — High-level system architecture: client → routes → engine → data.

Narration: route handlers stay thin and delegate to one Risk Engine that owns modelling and graph analytics. The engine reads financials from Postgres and relationships from Neo4j; in demo mode it builds a synthetic economy so the whole system runs with zero external services.

3 The Core Insight — Why a Graph

A tabular model sees only one company's own numbers; it cannot see that a healthy company's biggest supplier is collapsing. Kavach models the economy as a graph and uses **message passing** so each prediction is informed by its neighbourhood.

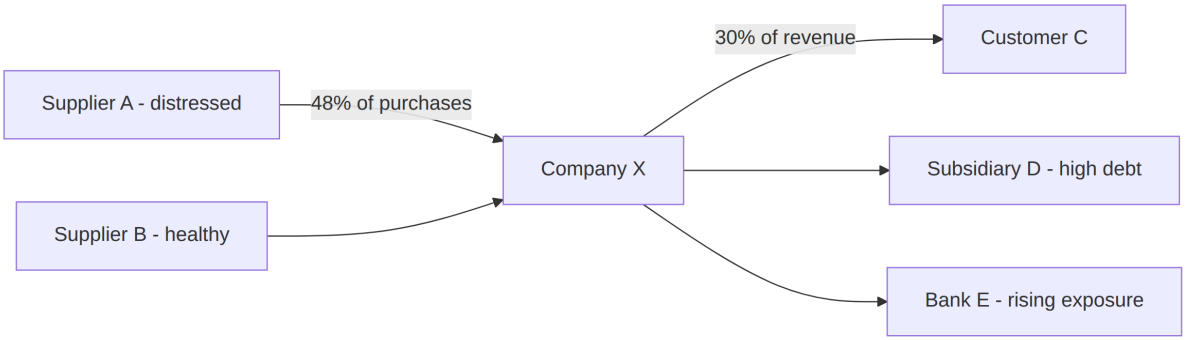


Diagram 2 — A healthy-looking company inside an unhealthy network.

Message passing: each company starts as its own feature vector; at each GNN layer it aggregates neighbours' vectors and updates its own. After two layers a company's embedding encodes its 1–2 hop neighbourhood, which a classifier head turns into a distress probability.

Guardrail: Don't claim Kavach connects to live real-world supply-chain feeds. The shipped system runs on a synthetic economy and is designed to accept real Postgres/Neo4j sources.

4 Mobile-Side Architecture

The Flutter app is a bottom-nav shell hosting workflow modes; each screen calls thin API routes through a single ApiClient.

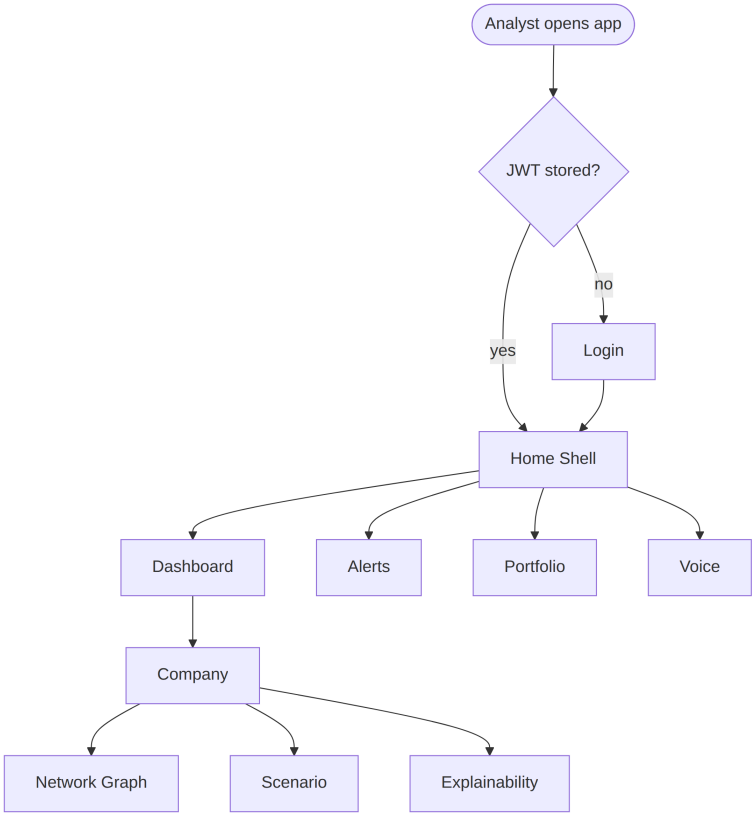


Diagram 3 — Mobile navigation and screen-to-API mapping.

Talking points

- State in a ChangeNotifier (AppState): auth token, current user, selected locale.
- JWT persisted in flutter_secure_storage; attached as a Bearer header by ApiClient.
- Network graph screen renders the ego graph with graphview, coloured by risk band.
- Localisation via Flutter ARB files (en/hi/ta), switchable at runtime from Settings.

5 Backend / API Layer

A thin-controller FastAPI app: routers validate input and delegate to the Risk Engine; they contain no modelling logic.

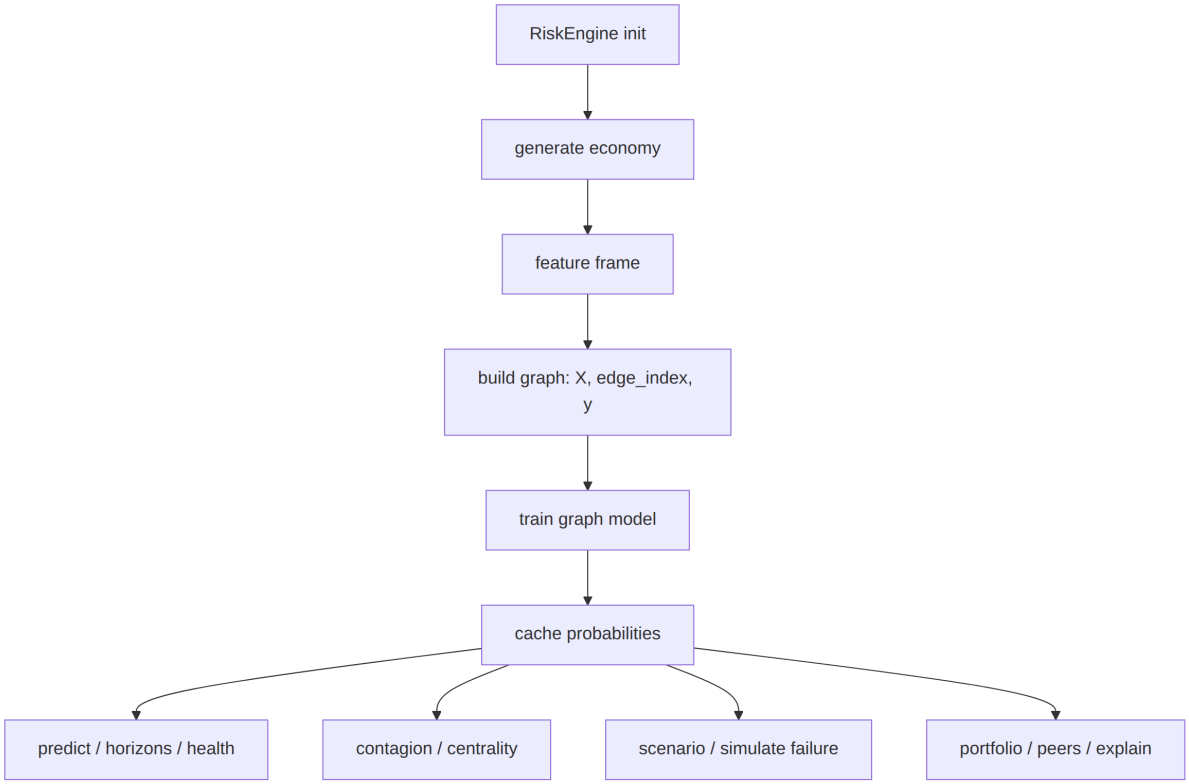


Diagram 4 — Risk Engine initialisation and method surface.

Talking points

- get_engine() is an lru_cache singleton — the model trains once per process.
- Auth is JWT (HS256) via python-jose; passwords hashed with pbkdf2_sha256.
- Postgres and Neo4j layers are optional (is_configured guards); unset ⇒ synthetic.
- Heavy routes degrade gracefully; the API is designed never to hard-fail a request.

6 The Risk Engine — Inference & Analytics Core

kavach_ml.serve.engine.RiskEngine is the heart of the system. On first use it generates the economy, builds the graph, trains the graph model, and caches predictions. Every product feature is a method on this class, guaranteeing all endpoints return numbers consistent with the same model and graph.

7 ML Pipeline — Data → Features → Graph → Models

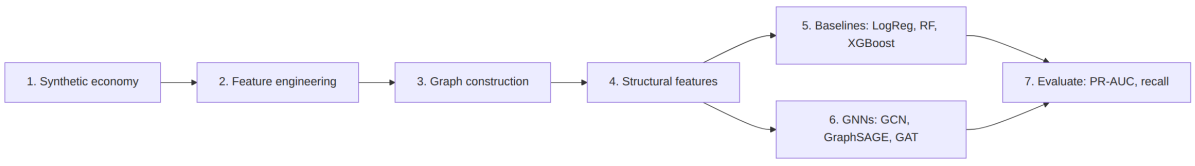


Diagram 5 — End-to-end ML pipeline.

Leakage-free experiment: the model's node features are a company's OWN financial ratios only. Graph structure reaches the GNN through edge_index message passing — not by injecting neighbour features into the tabular vector. Structural scalars (concentration, centrality) are kept separate for analytics and explainability, so the tabular-vs-graph comparison is fair.

The synthetic economy encodes distress from a company's own fundamentals AND its neighbours' health, plus explicit **network-induced distress** (healthy firms that fail because ≥2 direct counterparties fail) — the signal only a graph model can capture.

8 The Models — Baselines vs GCN / GraphSAGE / GAT

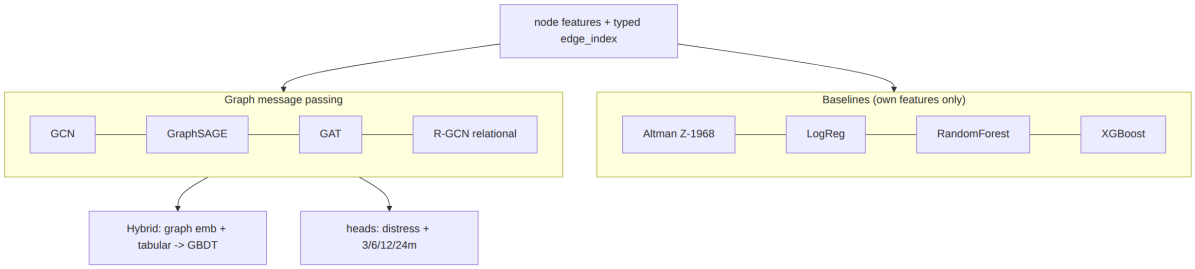


Diagram 6 — Tabular baselines vs graph models, shared prediction heads.

- **GCN:** degree-normalised neighbour averaging — a strong, simple baseline.
- **GraphSAGE:** samples and aggregates neighbours; scales to large graphs.
- **GAT:** learns attention weights, so a big supplier counts more than a tiny one; attention doubles as an explanation signal.
- **R-GCN (relational):** a separate transformation per relation type — the frontier model; tops the scoreboard on PR-AUC / KS.
- **Hybrid GNN + GBDT:** graph embedding + tabular features into a boosted tree — the SOTA recipe at scale (Relational-GNN paper, 2026).

Grounded in the literature (R-GCN, Schlichtkrull 2018; HGNN-EBP 2025; relational GNN + ensemble 2026 — see docs/RESEARCH.md). The repo ships production PyTorch Geometric models (GCN/GraphSAGE/GAT/RGCN), a NumPy reference GNN (SGC + relational + hybrid) that runs anywhere, and the classic Altman Z-Score (1968) baseline. Metrics follow the credit-risk literature: PR-AUC, KS, recall, precision@k.

> **Guardrail:** Don't over-claim the hybrid GNN+GBDT — it wins at scale (millions of nodes); on a few hundred synthetic nodes it under-fits versus a linear head. Report the regime honestly.

9 Handling Class Imbalance

Distress is the rare class, so accuracy is not a headline metric. The pipeline uses class weighting + focal loss on the GNN, scale_pos_weight for XGBoost, threshold tuning on validation, and PR-AUC / recall as the metrics that lead the scoreboard.

Guardrail: Describe this as per-model class weighting + focal loss + threshold tuning — not a full cost-sensitive learning framework, unless you extend it into one.

10 Explainable AI — Feature & Network Attribution

Kavach never shows a bare “78%”. It answers why.

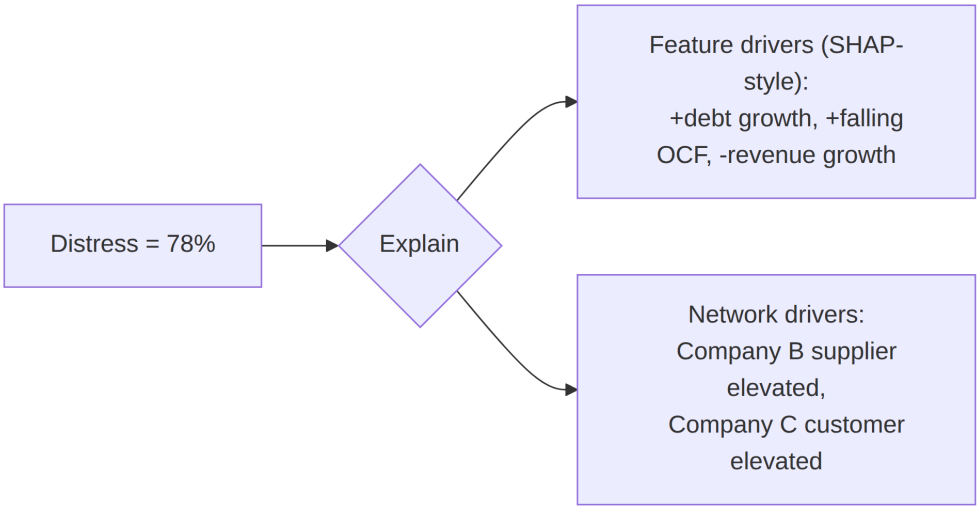


Diagram 7 — Two explanation channels: feature drivers and network drivers.

- **Tabular:** SHAP when available; a fast linear/importance fallback otherwise. Signed per-feature contributions + normalised share.
- **Graph:** ranks neighbours by influence = edge_weight × neighbour risk; with a trained GAT, reads attention weights or runs GNNExplainer.
- API can localise driver labels (e.g. /api/explain/{id}?lang=hi).

11 Network Analytics — Contagion, Centrality, Simulation

- **Contagion:** how exposed a firm is to distressed neighbours.
- **Centrality:** PageRank (power iteration) + degree — systemically important firms.
- **Concentration:** Herfindahl-Hirschman Index over supplier / customer edge weights.
- **What-if simulation:** simulate_failure propagates a collapse downstream with distance decay, returning the affected set + an impact level.

12 Scenario & Portfolio Intelligence

Scenario / stress test: recompute risk under a revenue shock, rate change and/or supplier failure — a closed-form perturbation of the base probability, with the supplier-failure term scaled by supplier concentration.

Portfolio risk: weight-average holdings' distress probabilities into one exposure, then surface shared-network concentration — e.g. “42% of your portfolio is linked to the same high-risk supplier network” — a graph-native insight a per-name model cannot produce.

13 Data Model — PostgreSQL + Neo4j

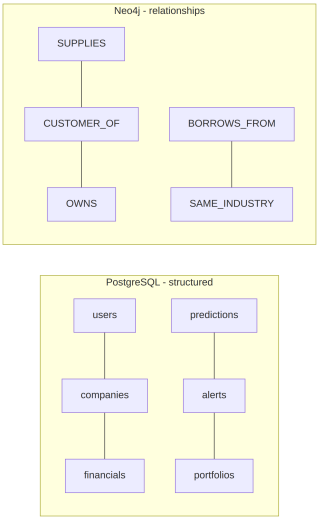


Diagram 8 — Two stores: structured financials (Postgres) and relationships (Neo4j).

Postgres holds users, companies, financial_statements, predictions (per-horizon columns), alerts, portfolios (JSONB) and a model_versions registry. Neo4j holds the relationship graph the GNN consumes; the 1-hop Cypher query feeds both the graph screen and the graph builder.

> **Guardrail:** Don't claim a relational DB was “rejected because it can't scale”. The honest framing: financials are tabular (Postgres), relationships are a graph (Neo4j), each used for its strength.

14 Voice & Multilingual Pipeline

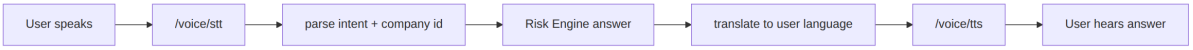


Diagram 9 — Voice loop: speak → STT → answer → translate → TTS → hear.

- STT: stub / Whisper / Azure / Google. TTS: stub / gTTS / Azure / Google — both degrade gracefully (stub returns valid silent WAV / echo).
- /api/voice/ask runs the full loop: transcribe → parse company id → answer → translate → speak.
- 12 Indian languages via /api/language/supported; UI localised through Flutter ARB files; translation service falls back builtin-glossary → passthrough.

15 Security & Resilience

• JWT (HS256), Bearer tokens, hashed passwords (pbkdf2_sha256).

• Pydantic input validation on every request body; bounded query params.

• Fail-soft: optional providers and databases degrade to safe defaults; the synthetic engine guarantees the API works standalone.

• CORS configurable; secrets via environment; /api/health for orchestrators.

Guardrail: The shipped auth store is in-memory with a seeded demo user; describe it as a JWT flow ready to be backed by Postgres/Firebase, not a hardened identity system.

16 Deployment

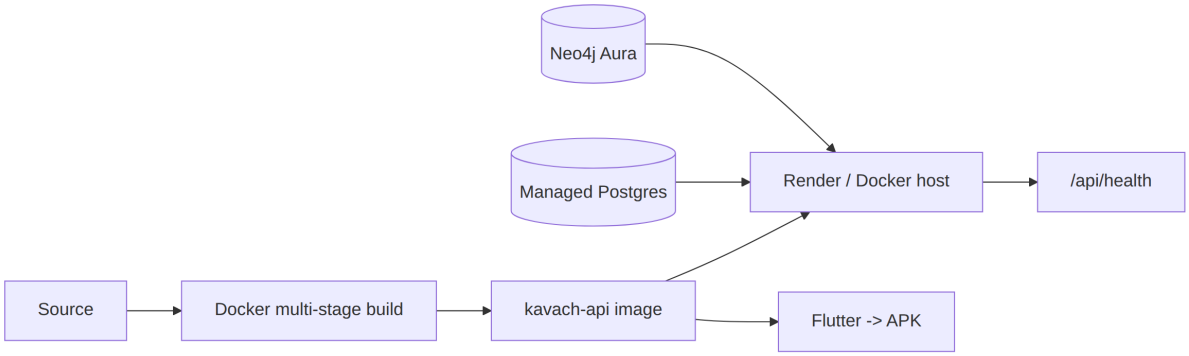


Diagram 10 — Build-to-deploy pipeline.

• Backend image: backend/Dockerfile multi-stage build; Uvicorn + /api/health HEALTHCHECK.

• Full local stack: docker compose up --build (Postgres + Neo4j + API).

• Managed deploy: render.yaml provisions web service + managed Postgres; Neo4j via Aura.

• Mobile: flutter build apk --release --dart-define=KAVACH_API_BASE=…

• CI/CD: GitHub Actions runs ML tests + comparison, API tests, Docker build, Flutter analyze.

17 Trade-offs & Known Weaknesses

Being candid here is a strong engineering signal.

• Default uses the NumPy reference GNN for portability; full PyG GCN/GraphSAGE/GAT needs torch installed.

• Data is synthetic; no real-data ETL in the repo yet (pipeline is source-agnostic).

• Auth user store is in-memory; should be Postgres/Firebase-backed for production.

- Scenario recomputation is a closed-form approximation, not full GNN re-inference.
- Contagion / failure simulation use heuristic decay, not a calibrated cascade model.
- No temporal modelling yet (temporal GNN is on the roadmap).

18 Interview Mental Model (Master Diagram)

Explain this one diagram and zoom into any box on a follow-up, and you can navigate almost any Kavach architecture question.

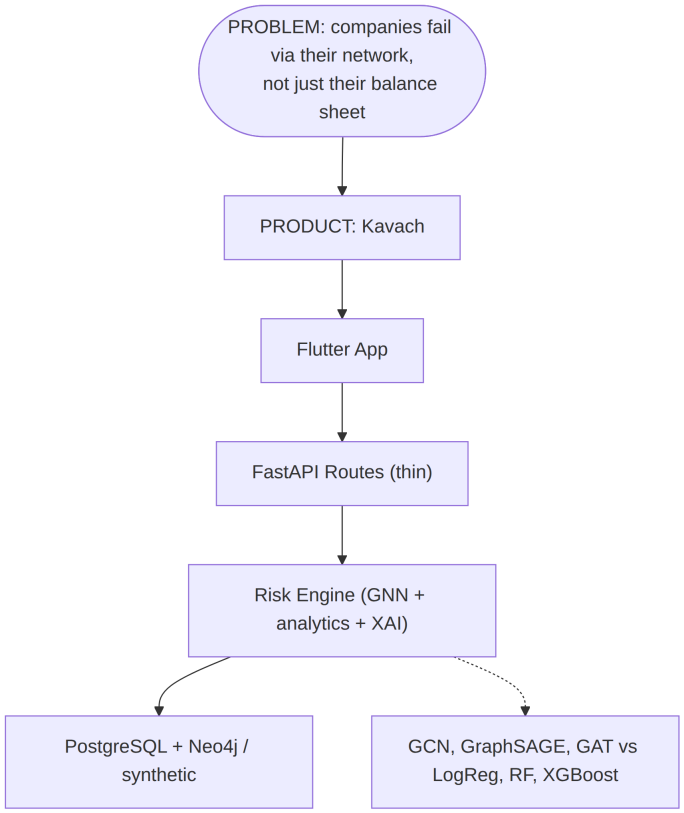


Diagram 11 — problem → product → app → API → engine → data.

19 Claims to Avoid Unless Verified in Code

- Don't claim live real-world data ingestion — it runs on a synthetic economy.
- Don't claim a relational DB was rejected for scale — each store is used for its strength.
- Don't claim the PyTorch GAT trains by default — the portable default is the NumPy GNN.
- Don't claim a production-grade identity system — auth is a JWT flow + seeded user store.
- Don't invent exact accuracy figures — quote the reproducible quick_demo.py scoreboard.

20 API Route Catalog

Auth	POST /api/auth/register · POST /api/auth/login
Companies	GET /api/companies · /companies/search · /companies/{id}
Predictions	GET /api/predictions/{id} · /multi-horizon · /health-score
Network	GET /network/{id}/graph · /contagion · POST /simulate-failure · GET /heatmap
Scenario	POST /api/scenario · GET /api/scenario/{id}/sweep
Portfolio	POST /api/portfolio/risk
Explain / Alerts	GET /api/explain/{id}?lang= · GET /api/alerts/early-warning
Language / Voice	GET /language/supported · POST /language/translate · /voice/tts · /voice/stt · /voice/ask
Ops	GET /api/ml/model-comparison · /api/ml/dataset-summary · /api/health

21 Rapid-Fire Q&A;

Q. In one sentence, what is Kavach?

A. A corporate financial-distress prediction platform that uses Graph Neural Networks to model supplier/customer/ownership/lending relationships, so a company's risk reflects its whole network — delivered as a Flutter app over FastAPI + Postgres + Neo4j.

Q. Why a GNN instead of XGBoost?

A. Tabular models see only a company's own financials; a GNN passes messages along relationship edges and captures contagion. We train both and show the graph model wins on PR-AUC and recall.

Q. How do you prove the graph actually helps?

A. Same stratified split for every model; node features are own-financials only; graph reaches the GNN via message passing, not injected features. Synthetic labels include network-induced distress tabular models can't capture; the scoreboard shows GraphSAGE/GCN topping PR-AUC.

Q. How do you handle class imbalance?

A. Class weighting + focal loss on the GNN, scale_pos_weight for XGBoost, threshold tuned on validation, and PR-AUC / recall as headline metrics — never raw accuracy.

Q. What is GAT and why does it matter?

A. Graph Attention Network learns how much weight to give each neighbour, so a big supplier counts more than a tiny one; the attention weights explain which connected company drove the risk.

Q. Why two databases?

A. Financials are naturally tabular (Postgres); relationships are naturally a graph (Neo4j). Each store does what it is best at.

Q. What happens if a provider or DB is down?

A. Everything degrades to safe defaults — stub audio, glossary/passthrough translation, synthetic engine — so the API never hard-fails. Fail-soft is deliberate.

Q. What would you improve first with more time?

A. Real-data ETL into Postgres/Neo4j, move auth to a persistent store, a temporal GNN for evolving relationships, and true scenario re-inference under perturbed inputs.